Lab Manual

The University of Faisalabad

By

Maham Nisar **2023-BS-AI-058**

Submitted to
Sir Samraiz

Bachelor of Science in Artificial Intelligence
DEPARTMENT OF COMPUTER SCIENCES

Table of Contents

Lab No: 1

(PROGRAMMING FUNDAMENTALS OF PYTHON)

Q1. Banking Transaction Validator with PIN:
Simulate a bank ATM.
User enters a 4-digit PIN (check if valid).
Display account balance and ask for withdrawal amount.
If valid → deduct + update balance.
If invalid attempts ≥ 3 → block card.
Use operators to compute service charges (2% per withdrawal)?

Code:

Q1: Banking Transaction Validator with PIN

[11]:
```
pin, balance, tries = "1234", 10000, 0
while tries < 3:
    if input("Enter PIN: ") == pin:
        amt = float(input("Withdraw: "))
        if amt*1.02 <= balance:
            balance -= amt*1.02
            print("Done! Balance:", balance)
        else:
            print("No funds.")
        break
    else:
        tries += 1
        print("Wrong PIN,", 3-tries, "tries left")
else:
    print("Card Blocked!")
```

```
Enter PIN:  1234
Withdraw:  1200
Done! Balance: 8776.0
```

Q2. Grocery Store Inventory System:

Maintain item names, stock, and prices in a dictionary of dictionaries.

User selects items with quantity.

If stock available, deduct from inventory.

Apply discount slabs (10%, 15%, 20%).

Print final bill with tax (5%).

Code:

Q2: Grocery Store Inventory System

[14]:

```
store = {
    "apple": {"price": 50, "stock": 20},
    "banana": {"price": 10, "stock": 50},
    "milk": {"price": 100, "stock": 10}
}

cart, total = {}, 0

while True:
    item = input("Enter item (or 'done'): ").lower()
    if item == "done": break
    if item in store:
        qty = int(input("Enter quantity: "))
        if qty <= store[item]["stock"]:
            store[item]["stock"] -= qty
            cost = store[item]["price"] * qty
            cart[item] = cart.get(item, 0) + cost
            total += cost
        else:
            print("❌ Not enough stock.")
    else:
        print("❌ Item not found.")

# Apply discount
if total >= 1000: disc = 0.20
elif total >= 500: disc = 0.15
elif total >= 200: disc = 0.10
else: disc = 0

discount = total * disc
tax = (total - discount) * 0.05
final = total - discount + tax

print("\n🛒 Final Bill:")
for i, cost in cart.items():
    print(f"{i} : {cost}")
print(f"Total: {total}, Discount: {discount}, Tax: {tax}, Final: {final}")
```

```
Enter item (or 'done'):  Apple
Enter quantity:  10
Enter item (or 'done'):  done

🛒 Final Bill:
apple : 500
Total: 500, Discount: 75.0, Tax: 21.25, Final: 446.25
```

Q3. Student Grading with Subject Weightage:

Input marks for 5 subjects where each subject has different weightage (stored in a tuple).

Compute weighted average.

Assign grade (A/B/C/D/Fail).

Also, display whether the student is eligible for scholarship (≥80% weighted).

Code:

Q3: Student Grading with Subject Weightage

```
[6]: weights = (0.2, 0.25, 0.15, 0.25, 0.15)  # weightage for 5 subjects
marks = [float(input(f"Marks for subject {i+1}: ")) for i in range(5)]

# Weighted average
weighted_avg = sum(m * w for m, w in zip(marks, weights))

# Grade assignment
if weighted_avg >= 90: grade = "A"
elif weighted_avg >= 75: grade = "B"
elif weighted_avg >= 60: grade = "C"
elif weighted_avg >= 50: grade = "D"
else: grade = "Fail"

print(f"\nWeighted %: {weighted_avg:.2f}")
print(f"Grade: {grade}")
print("Scholarship Eligible 🎓" if weighted_avg >= 80 else "Not Eligible")
```

```
Marks for subject 1:  50
Marks for subject 2:  60
Marks for subject 3:  70
Marks for subject 4:  80
Marks for subject 5:  90

Weighted %: 69.00
Grade: C
Not Eligible
```

Q4. BMI + Health Recommendation:

Extend BMI program:

Take weight, height, and age.

Compute BMI.

Based on BMI and age, give personalized health suggestions (e.g., diet plan for obese, exercise plan for underweight).

Code:

Q4: BMI + Health Recommendation

[18]:
```
w = float(input("Weight (kg): "))
h = float(input("Height (m): "))
age = int(input("Age: "))

bmi = w / (h*h)
print(f"\nBMI: {bmi:.2f}")

if bmi < 18.5:
    msg = "Underweight → Eat protein-rich food & light exercise."
elif bmi < 24.9:
    msg = "Normal → Maintain balanced diet & regular activity."
elif bmi < 29.9:
    msg = "Overweight → Control diet, add cardio exercises."
else:
    msg = "Obese → Strict diet plan + daily workout."

# Age-based note
if age < 18: note = " (Focus on growth, avoid harsh dieting.)"
elif age > 50: note = " (Add walking & regular health checkups.)"
else: note = ""

print("Suggestion:", msg + note)
```

```
Weight (kg):  50
Height (m):  10
Age:  20

BMI: 0.50
Suggestion: Underweight → Eat protein-rich food & light exercise.
```

Q5. Telecom Prepaid Recharge System:

Maintain call rate/min, SMS rate, and data/MB in a dictionary.

User selects usage (calls, SMS, data).

Deduct accordingly from balance.

If balance < 20% of initial → print recharge reminder.

Apply bonus balance if recharge > 1000.

Code:

Q5. Telecom Prepaid Recharge System

[24]:
```
rates = {"call": 2, "sms": 1, "data": 5}  # per min, per SMS, per MB
balance = float(input("Enter recharge amount: "))
init_bal = balance

if balance > 1000:  # bonus
    balance += 100
    print("🎁 Bonus 100 added! New balance:", balance)

while True:
    use = input("\nEnter usage (call/sms/data/exit): ").lower()
    if use == "exit": break
    if use in rates:
        qty = float(input("Enter units: "))
        cost = qty * rates[use]
        if cost <= balance:
            balance -= cost
            print(f"Used {qty} {use}, Cost={cost}, Balance={balance}")
            if balance < 0.2 * init_bal:
                print("⚠️ Low Balance! Please recharge.")
        else:
            print("❌ Not enough balance.")
    else:
        print("❌ Invalid option.")
```

```
Enter recharge amount:  500

Enter usage (call/sms/data/exit):  call
Enter units:  35
Used 35.0 call, Cost=70.0, Balance=430.0

Enter usage (call/sms/data/exit):  exit
```

Q6. E-Commerce Discount Engine with Membership Levels:

User inputs bill amount + membership type (Regular, Gold, Platinum).

Apply different discount rules (Gold +10%, Platinum +15%).

If bill > 10000, apply additional 5% flat.

Show itemized receipt with tax breakdown.

Code:

Q6: E-Commerce Discount Engine

```
[27]: bill = float(input("Enter bill amount: "))
      mem = input("Membership (Regular/Gold/Platinum): ").lower()

      # Membership discount
      disc = 0
      if mem == "gold": disc += 0.10
      elif mem == "platinum": disc += 0.15

      # Extra discount for big bill
      if bill > 10000: disc += 0.05

      discount = bill * disc
      subtotal = bill - discount
      tax = subtotal * 0.05
      final = subtotal + tax

      print("\n🧾 Receipt")
      print(f"Bill: {bill}")
      print(f"Discount ({disc*100}%): -{discount}")
      print(f"Subtotal: {subtotal}")
      print(f"Tax (5%): +{tax}")
      print(f"Final Payable: {final}")
```

```
Enter bill amount:  1000
Membership (Regular/Gold/Platinum):  Gold

🧾 Receipt
Bill: 1000.0
Discount (10.0%): -100.0
Subtotal: 900.0
Tax (5%): +45.0
Final Payable: 945.0
```

Q7. Hospital Emergency Unit with Multi-Condition Triage:

Take patient details: temperature, pulse, oxygen, blood pressure.

Classify into categories: Stable, Observation, Urgent, Critical.

If critical → suggest "Admit in ICU."

If urgent → suggest "Immediate doctor attention."

Use nested conditions.

Code:

Q7. Hospital Emergency Unit Triage

```
[30]: temp = float(input("Temperature (°C): "))
      pulse = int(input("Pulse rate: "))
      oxy = int(input("Oxygen level %: "))
      bp = int(input("Blood Pressure (systolic): "))

      if temp > 40 or oxy < 85 or bp < 80:
          status, advice = "Critical", "Admit in ICU!"
      elif temp > 38 or oxy < 90 or pulse > 120 or bp < 90:
          status, advice = "Urgent", "Immediate doctor attention!"
      elif temp > 37.5 or pulse > 100 or oxy < 94:
          status, advice = "Observation", "Keep under monitoring."
      else:
          status, advice = "Stable", "Normal care, no emergency."

      print(f"\nStatus: {status}\nAdvice: {advice}")
```

```
Temperature (°C):  120
Pulse rate:  30
Oxygen level %:  34
Blood Pressure (systolic):  50

Status: Critical
Advice: Admit in ICU!
```

Q8. Online Examination Grader with Negative Marking:

Take total marks, obtained marks, and number of wrong answers.

Deduct 0.25 per wrong answer.

Compute percentage.

Assign division + eligibility for scholarship.

If Fail → suggest "Repeat Exam."

Code:

Q8. Online Examination Grader

```
[33]: total = int(input("Total Marks: "))
      obt = float(input("Obtained Marks: "))
      wrong = int(input("Wrong Answers: "))

      # Negative marking
      obt -= wrong * 0.25
      perc = (obt / total) * 100

      # Division
      if perc >= 60: div = "First"
      elif perc >= 45: div = "Second"
      elif perc >= 33: div = "Third"
      else: div = "Fail"

      print(f"\nFinal Marks: {obt:.2f}/{total}")
      print(f"Percentage: {perc:.2f}%")
      print(f"Division: {div}")

      if perc >= 80: print("Scholarship Eligible 🎓")
      elif div == "Fail": print("❌ Repeat Exam")
```

```
Total Marks:  50
Obtained Marks:  45
Wrong Answers:  6

Final Marks: 43.50/50
Percentage: 87.00%
Division: First
Scholarship Eligible 🎓
```

Q9. Movie Ticket Booking with Age & Timing Policy:

User inputs: age, showtime, ticket type (Normal/3D/IMAX).

Apply child restrictions (No late-night for <12).

Apply senior citizen discount (30%).

Apply peak hour charges (10%).

Print final ticket price.

Code:

Q9. Movie Ticket Booking

```
[36]: prices = {"normal": 500, "3d": 700, "imax": 1000}

age = int(input("Enter age: "))
show = int(input("Showtime (24hr e.g. 21 for 9pm): "))
typ = input("Ticket type (Normal/3D/IMAX): ").lower()

if age < 12 and show >= 22:
    print("❌ Children under 12 not allowed for late-night shows.")
else:
    price = prices.get(typ, 500)

    if age >= 60: price *= 0.7          # 30% discount
    if 18 <= show <= 22: price *= 1.10  # peak hour

    print(f"Final Ticket Price: {price}")
```

```
Enter age:  23
Showtime (24hr e.g. 21 for 9pm):  9pm
```

Q10. Weather-based Outfit + Travel Suggestion:

Input temperature, weather (Rainy/Sunny/Cold), event type (Casual/Business/Party).

Suggest outfit + accessories.

If Rainy → umbrella; If Cold → gloves.

If event is Business + temperature > 35 → recommend "light formal suit."

Code:

Q10. Weather-based Outfit + Travel Suggestion

[39]:
```
temp = float(input("Temperature (°C): "))
weather = input("Weather (Rainy/Sunny/Cold): ").lower()
event = input("Event (Casual/Business/Party): ").lower()

if event == "business" and temp > 35:
    outfit = "Light formal suit"
elif event == "casual":
    outfit = "T-shirt & jeans"
elif event == "party":
    outfit = "Trendy outfit with accessories"
else:
    outfit = "Formal wear"

if weather == "rainy": outfit += " + Umbrella"
elif weather == "cold": outfit += " + Gloves"
else: outfit += " + Sunglasses"

print("\nSuggestion:", outfit)
```

```
Temperature (°C):  2
Weather (Rainy/Sunny/Cold):  Cold
Event (Casual/Business/Party):  Party

Suggestion: Trendy outfit with accessories + Gloves
```

Lab No: 02

(OOP in Python)

Question # 1: Create a `BankAccount` class that represents a bank account. It should have attributes for account number, account holder name, and balance. Include methods to deposit, withdraw, and display balance. Ensure withdrawals cannot exceed the available balance?

Code:

[2]:
```python
class BankAccount:
    def __init__(self, account_number, account_holder, balance=0.0):
        self.account_number = account_number
        self.account_holder = account_holder
        self.balance = balance

    def deposit(self, amount):
        if amount > 0:
            self.balance += amount
            print(f"Deposited: ${amount:.2f}")
        else:
            print("Deposit amount must be positive!")

    def withdraw(self, amount):
        if amount > self.balance:
            print("Insufficient balance! Withdrawal denied.")
        elif amount <= 0:
            print("Withdrawal amount must be positive!")
        else:
            self.balance -= amount
            print(f"Withdrawn: ${amount:.2f}")

    def display_balance(self):
        print(f"Account Holder: {self.account_holder}")
        print(f"Account Number: {self.account_number}")
        print(f"Current Balance: ${self.balance:.2f}")


# Example usage
account1 = BankAccount("123456789", "Maham Nisar", 5000)

account1.display_balance()
account1.deposit(1500)
account1.withdraw(2000)
account1.withdraw(6000)  # Will show insufficient balance
account1.display_balance()
```

OUTPUT:

```
Account Holder: Maham Nisar
Account Number: 123456789
Current Balance: $5000.00
Deposited: $1500.00
Withdrawn: $2000.00
Insufficient balance! Withdrawal denied.
Account Holder: Maham Nisar
Account Number: 123456789
Current Balance: $4500.00
```

Question # 2: Design a `Book` class with attributes like title, author, ISBN, and availability status. Create a `Library` class that can add books, search for books by title/author, check out books, and return books. Track which books are currently available.

Code:

[13]:
```python
class Book:
    def __init__(self, title, author, isbn):
        self.title = title
        self.author = author
        self.isbn = isbn
        self.available = True  # Book is available by default

    def __str__(self):
        status = "Available" if self.available else "Checked Out"
        return f"'{self.title}' by {self.author} (ISBN: {self.isbn}) - {status}"
```

[15]:
```
class Library:
    def __init__(self):
        self.books = []  # List to store all books in the library

    def add_book(self, book):
        self.books.append(book)
        print(f"Book '{book.title}' added to the library.")

    def search_by_title(self, title):
        found_books = [book for book in self.books if title.lower() in book.title.lower()]
        if found_books:
            print("\nSearch Results (by Title):")
            for book in found_books:
                print(book)
        else:
            print(f"No books found with title containing '{title}'.")

    def search_by_author(self, author):
        found_books = [book for book in self.books if author.lower() in book.author.lower()]
        if found_books:
            print("\nSearch Results (by Author):")
            for book in found_books:
                print(book)
        else:
            print(f"No books found by author '{author}'.")

    def check_out(self, isbn):
        for book in self.books:
            if book.isbn == isbn:
                if book.available:
                    book.available = False
                    print(f"You have checked out '{book.title}'.")
                else:
                    print(f"'{book.title}' is already checked out.")
                return
        print("Book not found in the library.")

    def return_book(self, isbn):
        for book in self.books:
            if book.isbn == isbn:
                if not book.available:
                    book.available = True
                    print(f"'{book.title}' has been returned.")
                else:
                    print(f"'{book.title}' was not checked out.")
                return
        print("Book not found in the library.")


# Example usage:
book1 = Book("The Great Gatsby", "F. Scott Fitzgerald", "12345")
book2 = Book("To Kill a Mockingbird", "Harper Lee", "67890")
book3 = Book("Python Programming", "John Zelle", "11111")

library = Library()
library.add_book(book1)
library.add_book(book2)
library.add_book(book3)

library.search_by_title("python")
library.check_out("11111")
library.return_book("11111")
library.search_by_author("lee")
```

OUTPUT:

```
Book 'The Great Gatsby' added to the library.
Book 'To Kill a Mockingbird' added to the library.
Book 'Python Programming' added to the library.

Search Results (by Title):
'Python Programming' by John Zelle (ISBN: 11111) - Available
You have checked out 'Python Programming'.
'Python Programming' has been returned.

Search Results (by Author):
'To Kill a Mockingbird' by Harper Lee (ISBN: 67890) - Available
```

Question # 3: Create a `Student` class with attributes for student ID, name, and a list of grades. Implement methods to add grades, calculate average grade, and determine the letter grade (A, B, C, etc.). Add a method to display student information with their performance summary.

Code:

[20]:

```python
class Student:
    def __init__(self, student_id, name):
        self.student_id = student_id
        self.name = name
        self.grades = []

    def add_grade(self, grade):
        if 0 <= grade <= 100:
            self.grades.append(grade)
            print(f"Grade {grade} added for {self.name}.")
        else:
            print("Invalid grade! Please enter a value between 0 and 100.")

    def calculate_average(self):
        if not self.grades:
            return 0
        return sum(self.grades) / len(self.grades)

    def get_letter_grade(self):
        avg = self.calculate_average()
        if avg >= 90:
            return 'A'
        elif avg >= 80:
            return 'B'
        elif avg >= 70:
            return 'C'
        elif avg >= 60:
            return 'D'
        else:
            return 'F'

    def display_info(self):
        print(f"\n--- Student Information ---")
        print(f"Student ID: {self.student_id}")
        print(f"Name: {self.name}")
        print(f"Grades: {self.grades}")
        print(f"Average Grade: {self.calculate_average():.2f}")
        print(f"Letter Grade: {self.get_letter_grade()}")
```

```python
# Example usage
student1 = Student("S001", "Maham Nisar")
student1.add_grade(85)
student1.add_grade(90)
student1.add_grade(78)

student1.display_info()
```

OUTPUT:

```
Grade 85 added for Maham Nisar.
Grade 90 added for Maham Nisar.
Grade 78 added for Maham Nisar.

--- Student Information ---
Student ID: S001
Name: Maham Nisar
Grades: [85, 90, 78]
Average Grade: 84.33
Letter Grade: B
```

Question # 4: Design a `MenuItem` class for food items (name, price, category) and an `Order` class that can add multiple items, calculate total cost, apply discounts, and generate receipts. Include tax calculation functionality?

Code:

[23]:
```python
class MenuItem:
    def __init__(self, name, price, category):
        self.name = name
        self.price = price
        self.category = category

    def __str__(self):
        return f"{self.name} ({self.category}) - ${self.price:.2f}"
```

[25]:

```
class Order:
    def __init__(self, tax_rate=0.1):  # Default 10% tax
        self.items = []
        self.tax_rate = tax_rate
        self.discount = 0  # Discount percentage

    def add_item(self, item):
        self.items.append(item)
        print(f"Added: {item.name} (${item.price:.2f})")

    def apply_discount(self, discount_percent):
        if 0 <= discount_percent <= 100:
            self.discount = discount_percent
            print(f"Discount of {discount_percent}% applied.")
        else:
            print("Invalid discount percentage! Must be between 0 and 100.")

    def calculate_subtotal(self):
        return sum(item.price for item in self.items)

    def calculate_tax(self):
        return self.calculate_subtotal() * self.tax_rate

    def calculate_total(self):
        subtotal = self.calculate_subtotal()
        discount_amount = subtotal * (self.discount / 100)
        tax = (subtotal - discount_amount) * self.tax_rate
        total = subtotal - discount_amount + tax
        return total

    def generate_receipt(self):
        print("\n--- Receipt ---")
        for item in self.items:
            print(f"{item.name} - ${item.price:.2f}")
        subtotal = self.calculate_subtotal()
        discount_amount = subtotal * (self.discount / 100)
        tax = (subtotal - discount_amount) * self.tax_rate
        total = self.calculate_total()

        print(f"\nSubtotal: ${subtotal:.2f}")
        print(f"Discount ({self.discount}%): -${discount_amount:.2f}")
        print(f"Tax ({self.tax_rate * 100}%): +${tax:.2f}")
        print(f"Total: ${total:.2f}")
        print("--------------------")


# Example usage
burger = MenuItem("Cheese Burger", 5.99, "Fast Food")
pizza = MenuItem("Pepperoni Pizza", 8.49, "Fast Food")
coffee = MenuItem("Cappuccino", 3.50, "Beverage")

order1 = Order(tax_rate=0.08)  # 8% tax
order1.add_item(burger)
order1.add_item(pizza)
order1.add_item(coffee)
order1.apply_discount(10)
order1.generate_receipt()
```

OUTPUT:

```
Added: Cheese Burger ($5.99)
Added: Pepperoni Pizza ($8.49)
Added: Cappuccino ($3.50)
Discount of 10% applied.

--- Receipt ---
Cheese Burger - $5.99
Pepperoni Pizza - $8.49
Cappuccino - $3.50

Subtotal: $17.98
Discount (10%): -$1.80
Tax (8.0%): +$1.29
Total: $17.48
--------------------
```

Question # 5: Create classes for `Vehicle` (base class) and derived classes `Car`, `Motorcycle`, and `Truck`. Each vehicle should have attributes like license plate, model, rental price per day, and availability. Implement a system to rent vehicles and calculate rental costs?

Code:

[32]:
```python
class Vehicle:
    def __init__(self, license_plate, model, rental_price_per_day):
        self.license_plate = license_plate
        self.model = model
        self.rental_price_per_day = rental_price_per_day
        self.available = True

    def rent_vehicle(self, days):
        if self.available:
            self.available = False
            cost = self.rental_price_per_day * days
            print(f"{self.model} ({self.license_plate}) rented for {days} days. Total cost: ${cost:.2f}")
            return cost
        else:
            print(f"{self.model} ({self.license_plate}) is not available.")
            return 0

    def return_vehicle(self):
        if not self.available:
            self.available = True
            print(f"{self.model} ({self.license_plate}) has been returned and is now available.")
        else:
            print(f"{self.model} ({self.license_plate}) was not rented.")

    def display_info(self):
        status = "Available" if self.available else "Rented"
        print(f"{self.model} ({self.license_plate}) - ${self.rental_price_per_day}/day - {status}")
```

[38]:
```python
class Car(Vehicle):
    def __init__(self, license_plate, model, rental_price_per_day, seats):
        super().__init__(license_plate, model, rental_price_per_day)
        self.seats = seats

    def display_info(self):
        super().display_info()
        print(f"Type: Car | Seats: {self.seats}")
```

[42]:
```python
class Motorcycle(Vehicle):
    def __init__(self, license_plate, model, rental_price_per_day, engine_capacity):
        super().__init__(license_plate, model, rental_price_per_day)
        self.engine_capacity = engine_capacity

    def display_info(self):
        super().display_info()
        print(f"Type: Motorcycle | Engine: {self.engine_capacity}cc")
```

[46]:
```python
class Truck(Vehicle):
    def __init__(self, license_plate, model, rental_price_per_day, load_capacity):
        super().__init__(license_plate, model, rental_price_per_day)
        self.load_capacity = load_capacity

    def display_info(self):
        super().display_info()
        print(f"Type: Truck | Load Capacity: {self.load_capacity} tons")


# Example usage
car1 = Car("ABC-123", "Toyota Corolla", 50, 5)
bike1 = Motorcycle("MTR-555", "Yamaha YZF-R3", 30, 300)
truck1 = Truck("TRK-909", "Ford F-150", 80, 5)

vehicles = [car1, bike1, truck1]

print("\n--- Vehicle Information ---")
for v in vehicles:
    v.display_info()
    print()

car1.rent_vehicle(3)
truck1.rent_vehicle(2)
car1.return_vehicle()
```

OUTPUT:

```
--- Vehicle Information ---
Toyota Corolla (ABC-123) - $50/day - Available
Type: Car | Seats: 5

Yamaha YZF-R3 (MTR-555) - $30/day - Available
Type: Motorcycle | Engine: 300cc

Ford F-150 (TRK-909) - $80/day - Available
Type: Truck | Load Capacity: 5 tons

Toyota Corolla (ABC-123) rented for 3 days. Total cost: $150.00
Ford F-150 (TRK-909) rented for 2 days. Total cost: $160.00
Toyota Corolla (ABC-123) has been returned and is now available.
```

Question # 6: Design an `Employee` class with attributes for employee ID, name, position, and salary. Create methods to calculate monthly salary, annual salary, and apply raises. Include functionality to track employee details and employment duration?

Code:

[49]:
```python
from datetime import date

class Employee:
    def __init__(self, emp_id, name, position, salary, hire_date):
        self.emp_id = emp_id
        self.name = name
        self.position = position
        self.salary = salary  # Annual salary
        self.hire_date = hire_date  # hire_date should be a date object (YYYY, MM, DD)

    def calculate_monthly_salary(self):
        return self.salary / 12

    def calculate_annual_salary(self):
        return self.salary

    def apply_raise(self, percent):
        if percent > 0:
            increase = self.salary * (percent / 100)
            self.salary += increase
            print(f"Salary increased by {percent}% (${increase:.2f}). New annual salary: ${self.salary:.2f}")
        else:
            print("Raise percentage must be positive.")

    def calculate_employment_duration(self):
        today = date.today()
        years = today.year - self.hire_date.year
        months = today.month - self.hire_date.month
        days = today.day - self.hire_date.day

        if days < 0:
            months -= 1
            days += 30
        if months < 0:
            years -= 1
            months += 12

        return years, months, days
```

```
    def display_info(self):
        years, months, days = self.calculate_employment_duration()
        print(f"\n--- Employee Details ---")
        print(f"Employee ID: {self.emp_id}")
        print(f"Name: {self.name}")
        print(f"Position: {self.position}")
        print(f"Annual Salary: ${self.salary:.2f}")
        print(f"Monthly Salary: ${self.calculate_monthly_salary():.2f}")
        print(f"Employment Duration: {years} years, {months} months, {days} days")


# Example usage
emp1 = Employee("E001", "Maham Nisar", "Software Engineer", 90000, date(2021, 5, 10))
emp1.display_info()

emp1.apply_raise(10)
emp1.display_info()
```

OUTPUT:

```
--- Employee Details ---
Employee ID: E001
Name: Maham Nisar
Position: Software Engineer
Annual Salary: $90000.00
Monthly Salary: $7500.00
Employment Duration: 4 years, 5 months, 4 days
Salary increased by 10% ($9000.00). New annual salary: $99000.00

--- Employee Details ---
Employee ID: E001
Name: Maham Nisar
Position: Software Engineer
Annual Salary: $99000.00
Monthly Salary: $8250.00
Employment Duration: 4 years, 5 months, 4 days
```

Question # 7: Create a `Product` class with attributes for product ID, name, price, and stock quantity. Design a `ShoppingCart` class that can add products, remove products, calculate total cost, and handle checkout process while updating inventory?

Code:

[54]:
```python
class Product:
    def __init__(self, product_id, name, price, stock_quantity):
        self.product_id = product_id
        self.name = name
        self.price = price
        self.stock_quantity = stock_quantity

    def update_stock(self, quantity):
        self.stock_quantity += quantity

    def __str__(self):
        return f"{self.name} (ID: {self.product_id}) - ${self.price:.2f}, Stock: {self.stock_quantity}"
```

[56]:
```python
class ShoppingCart:
    def __init__(self):
        self.items = {}  # Dictionary: product -> quantity

    def add_product(self, product, quantity):
        if quantity <= 0:
            print("Quantity must be greater than 0.")
            return

        if product.stock_quantity >= quantity:
            if product in self.items:
                self.items[product] += quantity
            else:
                self.items[product] = quantity
            product.stock_quantity -= quantity
            print(f"Added {quantity} x {product.name} to cart.")
        else:
            print(f"Not enough stock for {product.name}. Available: {product.stock_quantity}")

    def remove_product(self, product, quantity):
        if product in self.items:
            if quantity >= self.items[product]:
                removed_qty = self.items.pop(product)
                product.stock_quantity += removed_qty
                print(f"Removed all {removed_qty} x {product.name} from cart.")
            else:
                self.items[product] -= quantity
                product.stock_quantity += quantity
                print(f"Removed {quantity} x {product.name} from cart.")
        else:
            print(f"{product.name} is not in the cart.")

    def calculate_total(self):
        total = sum(product.price * quantity for product, quantity in self.items.items())
        return total

    def checkout(self):
        if not self.items:
            print("Cart is empty! Add products before checkout.")
            return

        total_cost = self.calculate_total()
        print("\n--- Checkout Summary ---")
        for product, quantity in self.items.items():
```

```
            print(f"{product.name} - {quantity} x ${product.price:.2f} = ${product.price * quantity:.2f}")
        print(f"\nTotal Amount Due: ${total_cost:.2f}")
        print("Thank you for your purchase!")
        self.items.clear()  # Empty cart after checkout


# Example usage
p1 = Product("P101", "Laptop", 800.00, 5)
p2 = Product("P102", "Headphones", 50.00, 10)
p3 = Product("P103", "Mouse", 25.00, 8)

print("\n--- Product List ---")
for p in [p1, p2, p3]:
    print(p)

cart = ShoppingCart()
cart.add_product(p1, 1)
cart.add_product(p2, 2)
cart.add_product(p3, 1)

print(f"\nTotal Cost: ${cart.calculate_total():.2f}")
cart.remove_product(p2, 1)
cart.checkout()

print("\n--- Updated Product Stock ---")
for p in [p1, p2, p3]:
    print(p)
```

OUTPUT:

```
--- Product List ---
Laptop (ID: P101) - $800.00, Stock: 5
Headphones (ID: P102) - $50.00, Stock: 10
Mouse (ID: P103) - $25.00, Stock: 8
Added 1 x Laptop to cart.
Added 2 x Headphones to cart.
Added 1 x Mouse to cart.

Total Cost: $925.00
Removed 1 x Headphones from cart.

--- Checkout Summary ---
Laptop - 1 x $800.00 = $800.00
Headphones - 1 x $50.00 = $50.00
Mouse - 1 x $25.00 = $25.00

Total Amount Due: $875.00
Thank you for your purchase!

--- Updated Product Stock ---
Laptop (ID: P101) - $800.00, Stock: 4
Headphones (ID: P102) - $50.00, Stock: 9
Mouse (ID: P103) - $25.00, Stock: 7
```

Question # 8: Design a `Patient` class to store patient information (ID, name, age, medical history) and an `Appointment` class to manage doctor appointments. Include functionality to schedule, cancel, and view appointments?

Code:

[59]:
```python
from datetime import datetime

class Patient:
    def __init__(self, patient_id, name, age, medical_history=None):
        self.patient_id = patient_id
        self.name = name
        self.age = age
        self.medical_history = medical_history if medical_history else []

    def add_medical_record(self, record):
        self.medical_history.append(record)
        print(f"Added medical record for {self.name}: {record}")

    def view_medical_history(self):
        print(f"\n--- Medical History of {self.name} ---")
        if not self.medical_history:
            print("No medical records found.")
        else:
            for record in self.medical_history:
                print(f"- {record}")

    def display_info(self):
        print(f"\nPatient ID: {self.patient_id}")
        print(f"Name: {self.name}")
        print(f"Age: {self.age}")
        print(f"Total Records: {len(self.medical_history)}")
```

[61]:
```python
class Appointment:
    def __init__(self):
        self.appointments = []  # List of all scheduled appointments

    def schedule_appointment(self, patient, doctor_name, date_time):
        appointment = {
            "patient_id": patient.patient_id,
            "patient_name": patient.name,
            "doctor_name": doctor_name,
            "date_time": date_time,
            "status": "Scheduled"
        }
        self.appointments.append(appointment)
        print(f"Appointment scheduled for {patient.name} with Dr. {doctor_name} on {date_time}.")

    def cancel_appointment(self, patient_id, date_time):
        for appointment in self.appointments:
            if appointment["patient_id"] == patient_id and appointment["date_time"] == date_time:
                appointment["status"] = "Canceled"
                print(f"Appointment for Patient ID {patient_id} on {date_time} has been canceled.")
                return
        print("No matching appointment found to cancel.")

    def view_appointments(self):
        print("\n--- All Appointments ---")
        if not self.appointments:
            print("No appointments scheduled.")
        else:
            for appt in self.appointments:
                print(f"Patient: {appt['patient_name']} | Doctor: {appt['doctor_name']} | "
                      f"Date: {appt['date_time']} | Status: {appt['status']}")
```

[63]:
```python
# Example usage
p1 = Patient("P001", "Maham Nisar", 25)
p2 = Patient("P002", "Ali Ahmed", 32, ["Diabetes", "High Blood Pressure"])

p1.add_medical_record("Routine Checkup - Normal")
p2.add_medical_record("Follow-up Visit - Stable Condition")

p1.display_info()
p2.view_medical_history()

# Manage Appointments
appt_system = Appointment()
appt_system.schedule_appointment(p1, "Dr. Fatima", "2025-10-16 10:00 AM")
appt_system.schedule_appointment(p2, "Dr. Ahmed", "2025-10-17 02:30 PM")

appt_system.view_appointments()
appt_system.cancel_appointment("P002", "2025-10-17 02:30 PM")
appt_system.view_appointments()
```

OUTPUT:

```
Added medical record for Maham Nisar: Routine Checkup - Normal
Added medical record for Ali Ahmed: Follow-up Visit - Stable Condition

Patient ID: P001
Name: Maham Nisar
Age: 25
Total Records: 1

--- Medical History of Ali Ahmed ---
- Diabetes
- High Blood Pressure
- Follow-up Visit - Stable Condition
Appointment scheduled for Maham Nisar with Dr. Dr. Fatima on 2025-10-16 10:00 AM.
Appointment scheduled for Ali Ahmed with Dr. Dr. Ahmed on 2025-10-17 02:30 PM.

--- All Appointments ---
Patient: Maham Nisar | Doctor: Dr. Fatima | Date: 2025-10-16 10:00 AM | Status: Scheduled
Patient: Ali Ahmed | Doctor: Dr. Ahmed | Date: 2025-10-17 02:30 PM | Status: Scheduled
Appointment for Patient ID P002 on 2025-10-17 02:30 PM has been canceled.

--- All Appointments ---
Patient: Maham Nisar | Doctor: Dr. Fatima | Date: 2025-10-16 10:00 AM | Status: Scheduled
Patient: Ali Ahmed | Doctor: Dr. Ahmed | Date: 2025-10-17 02:30 PM | Status: Canceled
```

Question # 9: Create a `User` class for a social media platform with attributes for username, email, friends list, and posts. Implement methods to add friends, create posts, and display user timeline. Include privacy settings for posts.

Code:

[69]:
```python
class Post:
    def __init__(self, content, privacy="Public"):
        self.content = content
        self.privacy = privacy  # "Public", "Friends", or "Private"

    def __str__(self):
        return f"[{self.privacy}] {self.content}"
```

[71]:
```python
class User:
    def __init__(self, username, email):
        self.username = username
        self.email = email
        self.friends = []  # List of User objects
        self.posts = []    # List of Post objects

    def add_friend(self, friend):
        if friend not in self.friends and friend != self:
            self.friends.append(friend)
            friend.friends.append(self)
            print(f"{self.username} and {friend.username} are now friends.")
        elif friend == self:
            print("You cannot add yourself as a friend.")
        else:
            print(f"{friend.username} is already your friend.")

    def create_post(self, content, privacy="Public"):
        if privacy not in ["Public", "Friends", "Private"]:
            print("Invalid privacy setting! Use 'Public', 'Friends', or 'Private'.")
            return
        post = Post(content, privacy)
        self.posts.append(post)
        print(f"Post created by {self.username} with privacy: {privacy}")

    def display_timeline(self, viewer=None):
        print(f"\n--- {self.username}'s Timeline ---")
        if not self.posts:
            print("No posts yet.")
            return

        for post in self.posts:
            # Show post if it's public, or if the viewer is a friend, or the viewer is the user
            if post.privacy == "Public" or viewer == self or (viewer in self.friends and post.privacy == "Friends"):
                print(post)
        print("--------------------------")
```

[73]:
```python
# Example usage
user1 = User("maham_nisar", "maham@example.com")
user2 = User("ali_ahmed", "ali@example.com")
user3 = User("fatima_khan", "fatima@example.com")

# Add friends
user1.add_friend(user2)

# Create posts with different privacy levels
user1.create_post("Loving this new app!", "Public")
user1.create_post("Hanging out with friends today.", "Friends")
user1.create_post("My personal diary entry.", "Private")

# View timelines
user1.display_timeline(viewer=user1)   # Owner sees all posts
user1.display_timeline(viewer=user2)   # Friend sees public + friends posts
user1.display_timeline(viewer=user3)   # Stranger sees only public posts
```

OUTPUT:

```
maham_nisar and ali_ahmed are now friends.
Post created by maham_nisar with privacy: Public
Post created by maham_nisar with privacy: Friends
Post created by maham_nisar with privacy: Private

--- maham_nisar's Timeline ---
[Public] Loving this new app!
[Friends] Hanging out with friends today.
[Private] My personal diary entry.
--------------------------

--- maham_nisar's Timeline ---
[Public] Loving this new app!
[Friends] Hanging out with friends today.
--------------------------

--- maham_nisar's Timeline ---
[Public] Loving this new app!
--------------------------
```

Question # 10: Design a `Room` class with room number, type (single/double/suite), price, and availability. Create a `Booking` class to handle room reservations, check-in/check-out dates, and calculate total stay cost?

Code:

[76]:
```python
from datetime import datetime
```

[78]:
```python
class Room:
    def __init__(self, room_number, room_type, price_per_night):
        self.room_number = room_number
        self.room_type = room_type  # "Single", "Double", "Suite"
        self.price_per_night = price_per_night
        self.available = True  # Available by default

    def mark_unavailable(self):
        self.available = False

    def mark_available(self):
        self.available = True

    def display_info(self):
        status = "Available" if self.available else "Booked"
        print(f"Room {self.room_number} | Type: {self.room_type} | Price: ${self.price_per_night:.2f}/night | {status}")
```

[80]:

```
class Booking:
    def __init__(self):
        self.bookings = []  # List to store all booking records

    def reserve_room(self, room, guest_name, check_in_date, check_out_date):
        if not room.available:
            print(f"Room {room.room_number} is not available.")
            return

        # Convert dates to datetime objects
        check_in = datetime.strptime(check_in_date, "%Y-%m-%d")
        check_out = datetime.strptime(check_out_date, "%Y-%m-%d")

        if check_out <= check_in:
            print("Error: Check-out date must be after check-in date.")
            return

        nights = (check_out - check_in).days
        total_cost = nights * room.price_per_night

        booking_record = {
            "guest_name": guest_name,
            "room_number": room.room_number,
            "room_type": room.room_type,
            "check_in": check_in_date,
            "check_out": check_out_date,
            "nights": nights,
            "total_cost": total_cost
        }

        room.mark_unavailable()
        self.bookings.append(booking_record)
        print(f"Booking confirmed for {guest_name} in Room {room.room_number} from {check_in_date} to {check_out_date}.")
        print(f"Total cost: ${total_cost:.2f}")
```

```
    def check_out(self, room_number):
        for booking in self.bookings:
            if booking["room_number"] == room_number:
                print(f"\nGuest {booking['guest_name']} has checked out from Room {room_number}.")
                print(f"Stay Summary: {booking['nights']} nights | Total Paid: ${booking['total_cost']:.2f}")
                room = next((r for r in all_rooms if r.room_number == room_number), None)
                if room:
                    room.mark_available()
                self.bookings.remove(booking)
                return
        print(f"No active booking found for Room {room_number}.")

    def view_all_bookings(self):
        print("\n--- All Bookings ---")
        if not self.bookings:
            print("No bookings found.")
        else:
            for booking in self.bookings:
                print(f"{booking['guest_name']} | Room {booking['room_number']} | "
                      f"{booking['check_in']} → {booking['check_out']} | "
                      f"${booking['total_cost']:.2f}")
```

[82]:
```python
# Example usage
room1 = Room(101, "Single", 80)
room2 = Room(102, "Double", 120)
room3 = Room(201, "Suite", 200)

all_rooms = [room1, room2, room3]

print("\n--- Room Information ---")
for r in all_rooms:
    r.display_info()

booking_system = Booking()
booking_system.reserve_room(room2, "Maham Nisar", "2025-10-15", "2025-10-18")
booking_system.reserve_room(room3, "Ali Ahmed", "2025-10-20", "2025-10-22")

booking_system.view_all_bookings()

booking_system.check_out(102)

print("\n--- Updated Room Status ---")
for r in all_rooms:
    r.display_info()
```

OUTPUT:

```
--- Room Information ---
Room 101 | Type: Single | Price: $80.00/night | Available
Room 102 | Type: Double | Price: $120.00/night | Available
Room 201 | Type: Suite | Price: $200.00/night | Available
Booking confirmed for Maham Nisar in Room 102 from 2025-10-15 to 2025-10-18.
Total cost: $360.00
Booking confirmed for Ali Ahmed in Room 201 from 2025-10-20 to 2025-10-22.
Total cost: $400.00

--- All Bookings ---
Maham Nisar | Room 102 | 2025-10-15 → 2025-10-18 | $360.00
Ali Ahmed | Room 201 | 2025-10-20 → 2025-10-22 | $400.00

Guest Maham Nisar has checked out from Room 102.
Stay Summary: 3 nights | Total Paid: $360.00

--- Updated Room Status ---
Room 101 | Type: Single | Price: $80.00/night | Available
Room 102 | Type: Double | Price: $120.00/night | Available
Room 201 | Type: Suite | Price: $200.00/night | Booked
```

Question # 11: Create `Course` classes (with code, name, credits, capacity) and a `Student` class that can register for courses. Implement functionality to check for schedule conflicts and ensure students don't exceed credit limits?

Code:

```
0]: class Product:
        def __init__(self, pid, name, quantity, price, supplier):
            self.pid = pid
            self.name = name
            self.quantity = quantity
            self.price = price
            self.supplier = supplier


    class Inventory:
        def __init__(self):
            self.products = []

        def add_product(self, product):
            self.products.append(product)
            print(f"{product.name} added to inventory.")

        def update_quantity(self, pid, qty):
            for p in self.products:
                if p.pid == pid:
                    p.quantity += qty
                    print(f"{p.name} quantity updated to {p.quantity}")
                    return
            print("Product not found!")

        def low_stock_alert(self):
            print("\n⚠️ Low Stock Products:")
            for p in self.products:
                if p.quantity < 5:
                    print(f"- {p.name} (Qty: {p.quantity})")


    # --- Test Code ---
    i = Inventory()
    p1 = Product(1, "Laptop", 4, 80000, "Dell")
    p2 = Product(2, "Mouse", 10, 500, "Logitech")

    i.add_product(p1)
    i.add_product(p2)

    i.update_quantity(1, 2)
    i.low_stock_alert()
```

Output:

```
Laptop added to inventory.
Mouse added to inventory.
Laptop quantity updated to 6

⚠️ Low Stock Products:
```

Question # 12: Design a `Product` class for items in inventory (ID, name, quantity, price, supplier) and an `Inventory` class to track stock levels, add new products, update quantities, and generate low-stock alerts.

Code:

```
class Product:
    def __init__(self, pid, name, quantity, price, supplier):
        self.pid = pid
        self.name = name
        self.quantity = quantity
        self.price = price
        self.supplier = supplier


class Inventory:
    def __init__(self):
        self.products = []

    def add_product(self, product):
        self.products.append(product)
        print(f"{product.name} added to inventory.")

    def update_quantity(self, pid, qty):
        for p in self.products:
            if p.pid == pid:
                p.quantity += qty
                print(f"{p.name} quantity updated to {p.quantity}")
                return
        print("Product not found!")

    def low_stock_alert(self):
        print("\n⚠️ Low Stock Products:")
        for p in self.products:
            if p.quantity < 5:
                print(f"- {p.name} (Qty: {p.quantity})")


# --- Test Code ---
i = Inventory()
p1 = Product(1, "Laptop", 4, 80000, "Dell")
p2 = Product(2, "Mouse", 10, 500, "Logitech")

i.add_product(p1)
i.add_product(p2)

i.update_quantity(1, 2)
i.low_stock_alert()
```

```
Laptop added to inventory.
Mouse added to inventory.
Laptop quantity updated to 6

⚠️ Low Stock Products:
```

Question # 13: Create `Movie` classes (title, genre, duration, showtimes) and a `Theater` class with seating arrangement. Implement a booking system that reserves seats and calculates ticket prices with different categories (standard, premium).

Code:

```
class Movie:
    def __init__(self, title, genre, duration, showtime):
        self.title = title
        self.genre = genre
        self.duration = duration
        self.showtime = showtime


class Theater:
    def __init__(self, name, seats):
        self.name = name
        self.seats = seats
        self.booked = 0

    def book_seat(self, num):
        if self.booked + num <= self.seats:
            self.booked += num
            print(f"{num} seat(s) booked successfully!")
        else:
            print("Not enough seats available!")

    def available_seats(self):
        return self.seats - self.booked


# --- Test Code ---
m1 = Movie("Avengers", "Action", 120, "7:00 PM")
t1 = Theater("Galaxy Cinema", 50)

print(f"🎥 Movie: {m1.title} | Show: {m1.showtime}")
print(f"Available Seats: {t1.available_seats()}")

t1.book_seat(5)
print(f"Seats left: {t1.available_seats()}")
```

Output:

```
🎥 Movie: Avengers | Show: 7:00 PM
Available Seats: 50
5 seat(s) booked successfully!
Seats left: 45
```

Question # 14: Design a `Member` class with personal details, membership type (basic/premium), and attendance records. Create a `Trainer` class and a system to schedule training sessions between members and trainers.

Code:

```
class Member:
    def __init__(self, name, membership_type):
        self.name = name
        self.membership_type = membership_type
        self.attendance = 0

    def mark_attendance(self):
        self.attendance += 1
        print(f"{self.name}'s attendance marked. Total: {self.attendance}")


class Trainer:
    def __init__(self, name):
        self.name = name
        self.sessions = []

    def schedule_session(self, member):
        self.sessions.append(member)
        print(f"Session scheduled for {member.name} with Trainer {self.name}")


# --- Test Code ---
m1 = Member("Ayesha", "Premium")
m2 = Member("Ali", "Basic")

t1 = Trainer("Hassan")

t1.schedule_session(m1)
t1.schedule_session(m2)

m1.mark_attendance()
m2.mark_attendance()
```

Output:

```
Session scheduled for Ayesha with Trainer Hassan
Session scheduled for Ali with Trainer Hassan
Ayesha's attendance marked. Total: 1
Ali's attendance marked. Total: 1
```

Question # 15: Extend the bank account system to include `Savings Account` and `Checking Account` classes with different interest rates and withdrawal rules. Implement fund transfer between accounts.

Code:

```
class BankAccount:
    def __init__(self, acc_no, holder, balance=0):
        self.acc_no = acc_no
        self.holder = holder
        self.balance = balance

    def deposit(self, amount):
        self.balance += amount
        print(f"Deposited: {amount}, New Balance: {self.balance}")

    def withdraw(self, amount):
        if amount <= self.balance:
            self.balance -= amount
            print(f"Withdrawn: {amount}, Remaining: {self.balance}")
        else:
            print("Insufficient balance!")

    def transfer(self, other, amount):
        if amount <= self.balance:
            self.balance -= amount
            other.balance += amount
            print(f"Transferred {amount} to {other.holder}")
        else:
            print("Transfer failed! Not enough balance.")


class SavingsAccount(BankAccount):
    def __init__(self, acc_no, holder, balance=0, interest_rate=0.03):
        super().__init__(acc_no, holder, balance)
        self.interest_rate = interest_rate

    def add_interest(self):
        interest = self.balance * self.interest_rate
        self.balance += interest
        print(f"Interest added: {interest}, New Balance: {self.balance}")


class CheckingAccount(BankAccount):
    def __init__(self, acc_no, holder, balance=0):
        super().__init__(acc_no, holder, balance)

    def withdraw(self, amount):
        fee = 10  # small transaction fee
        total = amount + fee
        if total <= self.balance:
            self.balance -= total
            print(f"Withdrawn: {amount} (Fee {fee}), Remaining: {self.balance}")
        else:
            print("Insufficient funds!")
```

```
# --- Test Code ---
s1 = SavingsAccount(101, "Ayesha", 1000)
c1 = CheckingAccount(102, "Ali", 1500)

s1.add_interest()
s1.transfer(c1, 200)
c1.withdraw(300)
```

Output:

```
Interest added: 30.0, New Balance: 1030.0
Transferred 200 to Ali
Withdrawn: 300 (Fee 10), Remaining: 1390
```

Question # 16: Create a `Pizza` class with size, crust type, and toppings. Design an ordering system that calculates prices based on selections and allows custom pizza creation with various topping combinations.

Code:

```python
class Pizza:
    def __init__(self, size, crust, toppings):
        self.size = size
        self.crust = crust
        self.toppings = toppings

    def calculate_price(self):
        base_price = 500 if self.size == "Small" else 800 if self.size == "Medium" else 1100
        topping_price = 50 * len(self.toppings)
        return base_price + topping_price


class Order:
    def __init__(self):
        self.pizzas = []

    def add_pizza(self, pizza):
        self.pizzas.append(pizza)
        print(f"{pizza.size} Pizza added to order!")

    def total_bill(self):
        total = sum(p.calculate_price() for p in self.pizzas)
        print(f"Total Bill: Rs {total}")


# --- Test Code ---
p1 = Pizza("Medium", "Cheese Burst", ["Olives", "Mushroom"])
p2 = Pizza("Large", "Thin Crust", ["Pepperoni", "Cheese", "Capsicum"])

order = Order()
order.add_pizza(p1)
order.add_pizza(p2)
order.total_bill()
```

Output:

```
Medium Pizza added to order!
Large Pizza added to order!
Total Bill: Rs 2150
```

Question # 17: Design classes for `Car` (model, year, color, price) and `Car Manufacturer` that can produce different car models with various features. Include quality control checks and production tracking.

Code:

```
class Car:
    def __init__(self, model, year, color, price):
        self.model = model
        self.year = year
        self.color = color
        self.price = price

    def show_info(self):
        print(f"{self.year} {self.color} {self.model} – Rs {self.price}")


class CarManufacturer:
    def __init__(self, name):
        self.name = name
        self.cars = []

    def produce_car(self, model, year, color, price):
        car = Car(model, year, color, price)
        self.cars.append(car)
        print(f"{self.name} produced a {model}")
        return car

    def show_all_cars(self):
        print(f"\nCars produced by {self.name}:")
        for car in self.cars:
            car.show_info()


# --- Test Code ---
m1 = CarManufacturer("Toyota")
m1.produce_car("Corolla", 2023, "White", 4500000)
m1.produce_car("Yaris", 2022, "Black", 3800000)

m1.show_all_cars()
```

Output:

```
Toyota produced a Corolla
Toyota produced a Yaris

Cars produced by Toyota:
2023 White Corolla – Rs 4500000
2022 Black Yaris – Rs 3800000
```

Question # 18: Create `Teacher` and `Student` classes, and a `Classroom` class that manages assignments, grades, and attendance. Implement functionality to track student performance and generate class reports.

Code:

```python
class Teacher:
    def __init__(self, name, subject):
        self.name = name
        self.subject = subject


class Student:
    def __init__(self, name):
        self.name = name
        self.grades = {}

    def add_grade(self, subject, grade):
        self.grades[subject] = grade

    def average_grade(self):
        if not self.grades:
            return 0
        return sum(self.grades.values()) / len(self.grades)


class Classroom:
    def __init__(self, name, teacher):
        self.name = name
        self.teacher = teacher
        self.students = []

    def add_student(self, student):
        self.students.append(student)
        print(f"{student.name} added to {self.name}")

    def show_report(self):
        print(f"\nClassroom: {self.name}")
        print(f"Teacher: {self.teacher.name} ({self.teacher.subject})")
        for s in self.students:
            print(f"{s.name} - Average: {s.average_grade():.2f}")


# --- Test Code ---
t1 = Teacher("Sir Ahmed", "Math")
c1 = Classroom("BSCS-1A", t1)

s1 = Student("Ayesha")
s2 = Student("Ali")

s1.add_grade("Math", 90)
s1.add_grade("Science", 85)
s2.add_grade("Math", 70)

c1.add_student(s1)
c1.add_student(s2)
c1.show_report()
```

```
Ayesha added to BSCS-1A
Ali added to BSCS-1A

Classroom: BSCS-1A
Teacher: Sir Ahmed (Math)
Ayesha - Average: 87.50
Ali - Average: 70.00
```

Question # 19: Design `Flight` classes (flight number, destination, departure time, capacity) and a `Passenger` class. Create a booking system that reserves seats and manages passenger manifests.

Code:

```
class Passenger:
    def __init__(self, name):
        self.name = name


class Flight:
    def __init__(self, flight_no, destination, capacity):
        self.flight_no = flight_no
        self.destination = destination
        self.capacity = capacity
        self.passengers = []

    def book_passenger(self, passenger):
        if len(self.passengers) < self.capacity:
            self.passengers.append(passenger)
            print(f"{passenger.name} booked on flight {self.flight_no}")
        else:
            print("Flight full!")

    def show_passengers(self):
        print(f"\nPassengers on flight {self.flight_no}:")
        for p in self.passengers:
            print("-", p.name)


# --- Test Code ---
f1 = Flight("PK101", "Karachi", 3)

p1 = Passenger("Ayesha")
p2 = Passenger("Ali")
p3 = Passenger("Sara")
p4 = Passenger("Bilal")

f1.book_passenger(p1)
f1.book_passenger(p2)
f1.book_passenger(p3)
f1.book_passenger(p4)  # Should say Flight full!

f1.show_passengers()
```

```
Ayesha booked on flight PK101
Ali booked on flight PK101
Sara booked on flight PK101
Flight full!

Passengers on flight PK101:
- Ayesha
- Ali
- Sara
```

Question # 20: Create classes for `Smart Device` (base class) and specific devices like `Smart Light`, `Thermostat`, and `Security Camera`. Implement a `Smart Home` class that can control all devices and create automation routines.

Code:

```
class SmartDevice:
    def __init__(self, name):
        self.name = name
        self.status = "OFF"

    def turn_on(self):
        self.status = "ON"
        print(f"{self.name} is ON")

    def turn_off(self):
        self.status = "OFF"
        print(f"{self.name} is OFF")


class SmartLight(SmartDevice):
    pass

class Thermostat(SmartDevice):
    pass

class SecurityCamera(SmartDevice):
    pass


class SmartHome:
    def __init__(self):
        self.devices = []

    def add_device(self, device):
        self.devices.append(device)
        print(f"{device.name} added to SmartHome")

    def turn_all_on(self):
        for d in self.devices:
            d.turn_on()

    def turn_all_off(self):
        for d in self.devices:
            d.turn_off()


# --- Test Code ---
l1 = SmartLight("Living Room Light")
t1 = Thermostat("Main Thermostat")
c1 = SecurityCamera("Front Door Camera")

home = SmartHome()
home.add_device(l1)
home.add_device(t1)
home.add_device(c1)

home.turn_all_on()
home.turn_all_off()
```

Output:

```
Living Room Light added to SmartHome
Main Thermostat added to SmartHome
Front Door Camera added to SmartHome
Living Room Light is ON
Main Thermostat is ON
Front Door Camera is ON
Living Room Light is OFF
Main Thermostat is OFF
Front Door Camera is OFF
```

Lab No: 03

(NUMPY IN PYTHON)

Question 1: A weather station records temperatures (°C) for 7 days:
[21.1, 23.4, 19.8, 25.0, 26.5, 24.1, 22.0]

Code:

```
import numpy as np

# The list of temperatures provided
temperatures_list = [21.1, 23.4, 19.8, 25.0, 26.5, 24.1, 22.0]

# Convert the list into a numpy array
temperatures_array = np.array(temperatures_list)

# Calculate the metrics
average_temperature = np.mean(temperatures_array)
maximum_temperature = np.max(temperatures_array)
minimum_temperature = np.min(temperatures_array)
temperature_difference = maximum_temperature - minimum_temperature

# Print the results
print(f"Numpy Array: {temperatures_array}")
print(f"Average Temperature: {average_temperature:.2f}°C")
print(f"Maximum Temperature: {maximum_temperature:.2f}°C")
print(f"Minimum Temperature: {minimum_temperature:.2f}°C")
print(f"Temperature Difference (Range): {temperature_difference:.2f}°C")
```

Output:

```
Numpy Array: [21.1 23.4 19.8 25.  26.5 24.1 22. ]
Average Temperature: 23.13°C
Maximum Temperature: 26.50°C
Minimum Temperature: 19.80°C
Temperature Difference (Range): 6.70°C
```

Question 2: A fruit store records daily sales of apples for 4 weeks:
Week 1: [12, 15, 14, 10, 9, 8, 7]
Week 2: [11, 12, 13, 9, 5, 8, 6]
Week 3: [7, 9, 12, 10, 11, 13, 12]
Week 4: [10, 11, 12, 7, 8, 9, 11]

Code:

```
import numpy as np

# Data provided
sales_data = [
    [12, 15, 14, 10, 9, 8, 7],
    [11, 12, 13, 9, 5, 8, 6],
    [7, 9, 12, 10, 11, 13, 12],
    [10, 11, 12, 7, 8, 9, 11]
]

# Task: Create a 4x7 NumPy array
sales_array = np.array(sales_data)
print(f"Sales Array:\n{sales_array}")
```

Output:

```
Sales Array:
[[12 15 14 10  9  8  7]
 [11 12 13  9  5  8  6]
 [ 7  9 12 10 11 13 12]
 [10 11 12  7  8  9 11]]
```

Question 3: Students received marks in a test:
[56, 78, 45, 90, 88, 76, 59, 62]

Code:

```
import numpy as np

marks_list = [56, 78, 45, 90, 88, 76, 59, 62]

# Convert to NumPy array
marks_array = np.array(marks_list)
print(f"NumPy array: {marks_array}")

# Add 5 bonus marks
bonus_marks = marks_array + 5
print(f"Marks with bonus: {bonus_marks}")

# Count students scoring > 60
count_above_60 = np.sum(bonus_marks > 60)
print(f"Count above 60: {count_above_60}")

# Find median and standard deviation
median_marks = np.median(bonus_marks)
std_dev_marks = np.std(bonus_marks)
print(f"Median: {median_marks}")
print(f"Standard Deviation: {std_dev_marks}")
```

Output:

```
NumPy array: [56 78 45 90 88 76 59 62]
Marks with bonus: [61 83 50 95 93 81 64 67]
Count above 60: 7
Median: 74.0
Standard Deviation: 15.105876340020794
```

Question 4: A smartwatch stores daily step-counts for a user for 30 days.

Code:

```python
import numpy as np

# Generate a random NumPy array of shape (30,) with values between 2000 and 15000
# We use 15001 as np.random.randint excludes the high value
step_counts_flat = np.random.randint(2000, 15001, size=(30,))

# Reshape the 1D array into a 5x6 matrix
step_counts_matrix = step_counts_flat.reshape(5, 6)

print(f"Step counts matrix (5x6):\n{step_counts_matrix}\n")

# Find:

# Overall average steps
# The singular "Weekly average steps" likely refers to the overall average over the weeks
overall_avg_steps = np.mean(step_counts_flat)
print(f"Overall average steps: {overall_avg_steps:.2f}\n")

# Day with maximum steps (1-based index)
# np.argmax finds the index of the max value in the flattened array
max_day_index_flat = np.argmax(step_counts_flat)
max_day_number = max_day_index_flat + 1
print(f"Day with maximum steps (1-based index): {max_day_number}\n")

# Number of days user crossed 10,000 steps
days_crossed_10k = np.sum(step_counts_flat > 10000)
print(f"Number of days user crossed 10,000 steps: {days_crossed_10k}\n")
```

✓ 0.0s

Output:

```
Step counts matrix (5x6):
[[ 2853  2198 13059  6500  3775 12780]
 [ 8738 11632 14121 14147  9569  9685]
 [12237  8979 13440  9450  5006 10111]
 [ 7795  6267 11129  8234  2081  5377]
 [ 3698 12583  3523  5875  8671 11832]]

Overall average steps: 8511.50

Day with maximum steps (1-based index): 10

Number of days user crossed 10,000 steps: 11
```

Question 5: Heart rate readings (in BPM) for 10 patients recorded 4 times per day:

- Use array shape = **(10, 4)**

Code:

```
#Step 1: Initialize the NumPy array
import numpy as np
heart_rates = np.random.randint(60, 150, (10, 4))
print(f"Generated Heart Rate Array:\n{heart_rates}\n")
```

✓ 0.0s

```
Generated Heart Rate Array:
[[129 103 102  94]
 [ 84 122  83 126]
 [109  65 138 138]
 [103  85  88 115]
 [ 83 137 132 116]
 [104  94  92 129]
 [119  69  86  68]
 [133  68  93  96]
 [ 93 109 131  85]
 [ 86  85 125  68]]
```

```
#Step 2: Compute daily averages and identify abnormal readings
print("--- Results Per Patient ---")
for i, rates in enumerate(heart_rates):
    avg = np.mean(rates)
    abnormal = rates[rates > 120]
    print(f"Patient {i+1}:")
    print(f"  Daily Average BPM: {avg:.2f}")
    print(f"  Abnormal Readings (>120 BPM): {abnormal if abnormal.size > 0 else 'None'}")
    print("-" * 20)
```

✓ 0.0s

```
--- Results Per Patient ---
Patient 1:
  Daily Average BPM: 107.00
  Abnormal Readings (>120 BPM): [129]
--------------------
Patient 2:
  Daily Average BPM: 103.75
  Abnormal Readings (>120 BPM): [122 126]
--------------------
Patient 3:
  Daily Average BPM: 112.50
  Abnormal Readings (>120 BPM): [138 138]
--------------------
Patient 4:
  Daily Average BPM: 97.75
  Abnormal Readings (>120 BPM): None
--------------------
Patient 5:
  Daily Average BPM: 117.00
  Abnormal Readings (>120 BPM): [137 132]
--------------------
Patient 6:
  Daily Average BPM: 104.75
  Abnormal Readings (>120 BPM): [129]
--------------------
...
Patient 10:
  Daily Average BPM: 91.00
  Abnormal Readings (>120 BPM): [125]
--------------------
```

Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...

```python
#Step 3: Compute overall maximum and minimum
overall_max = np.max(heart_rates)
overall_min = np.min(heart_rates)
print("--- Overall Results ---")
print(f"Overall Maximum Heart Rate: {overall_max} BPM")
print(f"Overall Minimum Heart Rate: {overall_min} BPM")
```

✓ 0.0s

```
--- Overall Results ---
Overall Maximum Heart Rate: 138 BPM
Overall Minimum Heart Rate: 65 BPM
```

Question 6: Two branches of a store record monthly sales (in thousands):

- Which branch performed better overall?

Code:

```
import numpy as np

# Monthly sales data in thousands
branch_a = [122, 135, 150, 160, 155, 140]
branch_b = [110, 130, 148, 165, 158, 145]

# 1. Compute the month-by-month difference between branches (Branch A - Branch B)
# This assumes the lists are of the same length and order matters
sales_difference = [a - b for a, b in zip(branch_a, branch_b)]

# 2. Calculate average monthly sales of each branch
average_a = np.mean(branch_a)
average_b = np.mean(branch_b)
```

```
# 3. Compute the correlation between A and B
# np.corrcoef returns a correlation matrix, the value at [0, 1] is the coefficient
correlation_matrix = np.corrcoef(branch_a, branch_b)
correlation_coefficient = correlation_matrix[0, 1]

# Display results
print(f"Branch A Sales (in thousands): {branch_a}")
print(f"Branch B Sales (in thousands): {branch_b}")
print("-" * 40)
print(f"Monthly Sales Difference (A - B): {sales_difference} (in thousands)")
print(f"Average Monthly Sales for Branch A: ${average_a:.2f}k")
print(f"Average Monthly Sales for Branch B: ${average_b:.2f}k")
print(f"Correlation Coefficient between A and B: {correlation_coefficient:.4f}")
print("-" * 40)
```

```
# 4. Determine which branch performed better overall
if average_a > average_b:
    print(f"Overall, Branch A performed better with higher average sales.")
elif average_b > average_a:
    print(f"Overall, Branch B performed better with higher average sales.")
else:
    print(f"Branch A and Branch B performed equally well on average.")
```

Output:

```
Branch A Sales (in thousands): [122, 135, 150, 160, 155, 140]
Branch B Sales (in thousands): [110, 130, 148, 165, 158, 145]
----------------------------------------
Monthly Sales Difference (A - B): [12, 5, 2, -5, -3, -5] (in thousands)
Average Monthly Sales for Branch A: $143.67k
Average Monthly Sales for Branch B: $142.67k
Correlation Coefficient between A and B: 0.9812
----------------------------------------
Overall, Branch A performed better with higher average sales.
```

Question 7: Given a 4×4 grayscale image:
[[100, 120, 130, 90],
[80 , 110, 140, 100],
[70 , 90 , 120, 130],
[110, 140, 150, 160]]

Code:

```
import numpy as np

# Original image data
image_data = [[100, 120, 130, 90],
              [80, 110, 140, 100],
              [70, 90, 120, 130],
              [110, 140, 150, 160]]
```

```
# Convert to NumPy array
img_array = np.array(image_data, dtype=np.float64)

# --- Task related calculations ---

# Compute average brightness before transformation
avg_brightness_before = np.mean(img_array)
print(f"Average brightness before: {avg_brightness_before}")

# Normalize pixel values (0-1)
normalized_img = img_array / 255.0
# print(f"Normalized image array: \n{normalized_img}") # Optional print

# Apply a transformation: new_pixel = pixel * 1.2 on original scale data
transformed_img_array = img_array * 1.2

# Clip all values > 255 (limits values to the 0-255 range)
clipped_img_array = np.clip(transformed_img_array, 0, 255)

# Compute average brightness after transformation (on clipped array)
avg_brightness_after = np.mean(clipped_img_array)
print(f"Average brightness after: {avg_brightness_after}")
```

Output:

```
Average brightness before: 115.0
Average brightness after: 138.0
```

Question 8: Dataset contains 100 samples with 4 features each?

Code:

```python
import numpy as np

# 1. Generate a 100x4 matrix with values 1-100
# Reshaping a sequence of 400 values to fit a 100x4 matrix
data = np.linspace(1, 100, 400).reshape(100, 4)

# 2. Standardize the dataset: x_scaled = (x - mean) / std
# axis=0 ensures we calculate mean and std for each feature (column)
mean = np.mean(data, axis=0)
std = np.std(data, axis=0)
x_scaled = (data - mean) / std

# 3. Perform and verify results
col_mean = np.mean(x_scaled, axis=0)
col_var = np.var(x_scaled, axis=0)

print(f"Column-wise Mean (Expected ≈ 0): \n{col_mean}")
print(f"Column-wise Variance (Expected ≈ 1): \n{col_var}")

# Verification using NumPy functions
np.testing.assert_allclose(col_mean, 0, atol=1e-15)
np.testing.assert_allclose(col_var, 1, atol=1e-15)
print("\nVerification Successful: Results match expectations.")
```

Output:

```
Column-wise Mean (Expected ≈ 0):
[ 1.37667655e-16  2.35367281e-16  2.39808173e-16 -3.24185123e-16]
Column-wise Variance (Expected ≈ 1):
[1. 1. 1. 1.]

Verification Successful: Results match expectations.
```

Question 9: Simulate traffic density (cars per minute) recorded by 6 sensors over 24 hours.

code:

```
import numpy as np

# Set a random seed for reproducibility
np.random.seed(42)

# Generate array shape (24, 6) with values 10-60 cars/min
density_data = np.random.randint(10, 61, size=(24, 6))

# Find hourly average
hourly_average = np.mean(density_data, axis=1)
# print(f"Hourly averages: \n{hourly_average}") # Optional print

# Identify the busiest sensor
total_density_per_sensor = np.sum(density_data, axis=0)
busiest_sensor_index = np.argmax(total_density_per_sensor)
# Using 1-based indexing for user clarity
busiest_sensor_id = busiest_sensor_index + 1
busiest_sensor_total = total_density_per_sensor[busiest_sensor_index]

print(f"Busiest sensor is Sensor {busiest_sensor_id} with total density {busiest_sensor_total}")

# Apply a threshold (traffic cap)
capped_density_data = np.clip(density_data, None, 50)
# print(f"Capped data: \n{capped_density_data}") # Optional print

# Compute energy usage: energy = Σ(sensor_density × 0.5)
energy_usage = np.sum(capped_density_data * 0.5)
print(f"Total energy usage: {energy_usage}")
```

Output:

```
Busiest sensor is Sensor 2 with total density 885
Total energy usage: 2401.5
```

Question 10: You are simulating forces in a mechanical system.

- Compute transformed forces: F_new = F × T
- Compute magnitude of each force vector
- Identify the component experiencing highest transformed force
- Perform eigenvalue decomposition of T

Code:

```
import numpy as np

# Force vectors applied to 5 components
F = np.array([
    [5, 2, 1],
    [3, 1, 4],
    [6, 3, 2],
    [4, 2, 5],
    [7, 1, 3]
])
```

```python
# Transformation matrix
T = np.array([
    [2, 1, 0],
    [1, 3, 1],
    [0, 2, 2]
])

# Compute transformed forces: F_new = F x T
F_new = F @ T
print(f"Transformed Forces:\n{F_new}")

# Compute magnitude of each transformed force vector
magnitudes = np.linalg.norm(F_new, axis=1)
print(f"Magnitudes:\n{magnitudes}")
```

```python
# Identify the component experiencing highest transformed force
highest_force_index = np.argmax(magnitudes)
highest_force_magnitude = magnitudes[highest_force_index]
highest_force_component_id = highest_force_index + 1
print(f"Highest Force Component: {highest_force_component_id}, Magnitude: {highest_force_magnitude}")

# Perform eigenvalue decomposition of T
eigenvalues, eigenvectors = np.linalg.eig(T)
print(f"Eigenvalues of T:\n{eigenvalues}")
print(f"Eigenvectors of T:\n{eigenvectors}")
```

Output:

```
Transformed Forces:
[[12 13  4]
 [ 7 14  9]
 [15 19  7]
 [10 20 12]
 [15 16  7]]
Magnitudes:
[18.13835715 18.05547009 25.19920634 25.37715508 23.02172887]
Highest Force Component: 4, Magnitude: 25.37715508089904
Eigenvalues of T:
[4.30277564 2.         0.69722436]
Eigenvectors of T:
[[-3.11546502e-01  7.07106781e-01  3.86413754e-01]
 [-7.17421694e-01  2.70737023e-17 -5.03410424e-01]
 [-6.23093003e-01 -7.07106781e-01  7.72827507e-01]]
```

Lab No: 04

(SEABORN AND SCIKIT-LEARN)

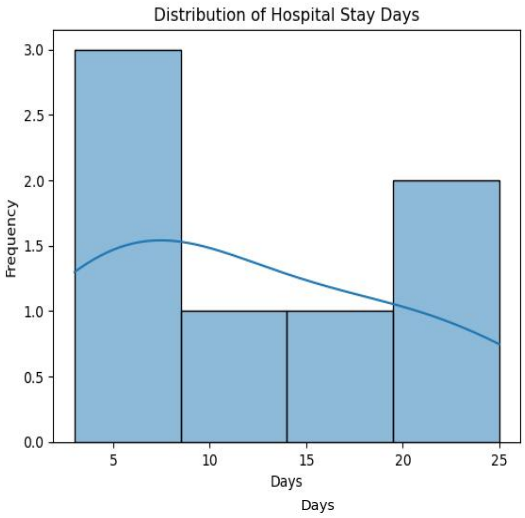
Q1. Patient Stay Analysis (Healthcare)
You are given a dataset containing age, disease_type, and hospital_stay_days.
Write a Python program using **Seaborn** to:

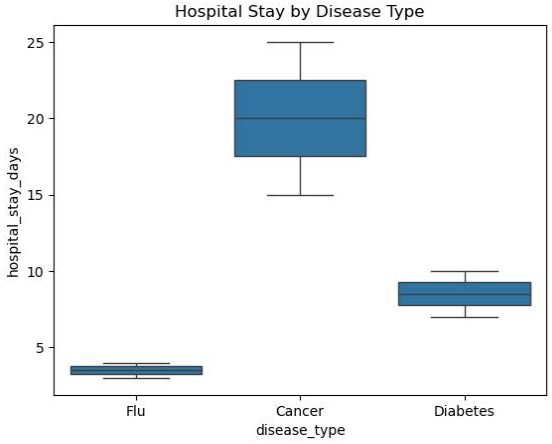
- Plot the distribution of hospital stay days
- Compare hospital stay across different disease types
- Identify which disease has the highest median stay

Code:

```
import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd

# Example dataset
data = pd.DataFrame({
    'age':[25,40,60,35,50,45,70],
    'disease_type':['Flu','Cancer','Cancer','Diabetes','Flu','Diabetes','Cancer'],
    'hospital_stay_days':[3,15,20,7,4,10,25]
})

# Distribution of hospital stay days
sns.histplot(data['hospital_stay_days'], kde=True)
plt.title("Distribution of Hospital Stay Days")
plt.xlabel("Days")
plt.ylabel("Frequency")
plt.show()

# Compare hospital stay across disease types
sns.boxplot(x='disease_type', y='hospital_stay_days', data=data)
plt.title("Hospital Stay by Disease Type")
plt.show()

# Identify disease with highest median stay
median_stays = data.groupby('disease_type')['hospital_stay_days'].median()
print("Disease with highest median stay:", median_stays.idxmax())
```

Output:






```
Disease with highest median stay: Cancer
```

Q2. Sales Distribution Comparison (Business)

A retail dataset contains region and monthly_sales.

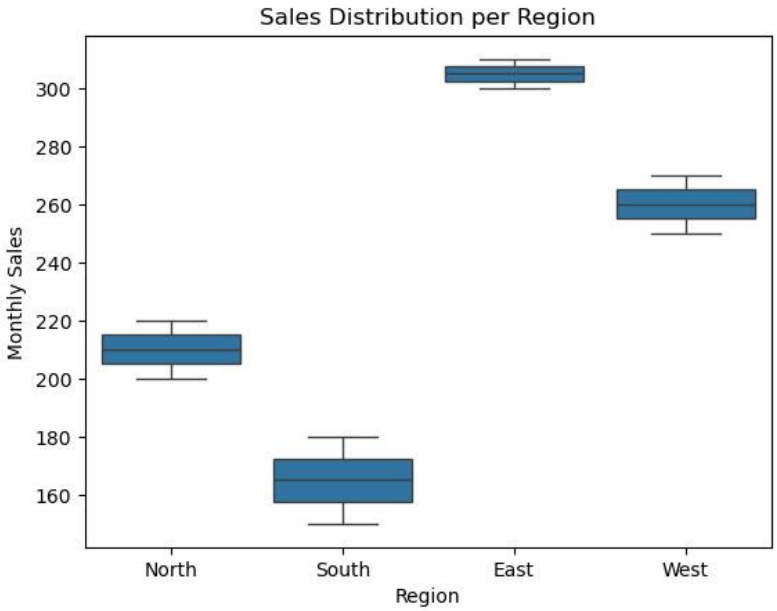
Write code to:

- Visualize sales distribution per region using Seaborn
- Detect regions with high sales variability
- Label the axes and add a title

Code:

```python
import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd

sales = pd.DataFrame({
    'region':['North','South','East','West','North','South','East','West'],
    'monthly_sales':[200,150,300,250,220,180,310,270]
})

# Sales distribution per region
sns.boxplot(x='region', y='monthly_sales', data=sales)
plt.title("Sales Distribution per Region")
plt.xlabel("Region")
plt.ylabel("Monthly Sales")
plt.show()

# Detect regions with high variability
variability = sales.groupby('region')['monthly_sales'].std()
print("Regions with high variability:\n", variability.sort_values(ascending=False))
```

Output:




```
Regions with high variability:
 region
South    21.213203
North    14.142136
West     14.142136
East      7.071068
Name: monthly_sales, dtype: float64
```

Q3. Disease Classification System

Patient data includes glucose, BMI, age, and outcome.

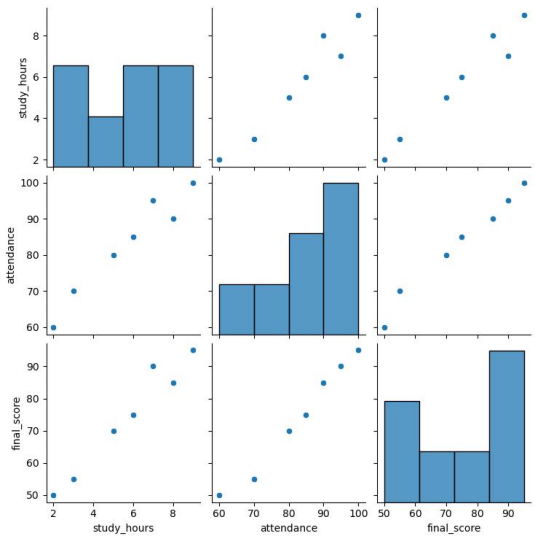
Write code to:

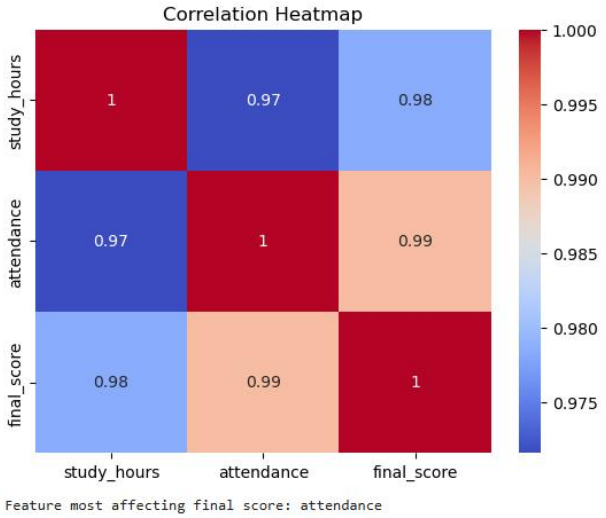
- Normalize the data
- Train a Decision Tree classifier
- Evaluate precision, recall, and F1-score

Code:

```
students = pd.DataFrame({
    'study_hours':[2,5,8,3,6,7,9],
    'attendance':[60,80,90,70,85,95,100],
    'final_score':[50,70,85,55,75,90,95]
})

# Pair plot
sns.pairplot(students)
plt.show()

# Correlation heatmap
corr = students.corr()
sns.heatmap(corr, annot=True, cmap='coolwarm')
plt.title("Correlation Heatmap")
plt.show()

print("Feature most affecting final score:", corr['final_score'].drop('final_score').idxmax())
```

Output:





Q4. Customer Segmentation Visualization

Customer data includes age, annual_income, and spending_score.

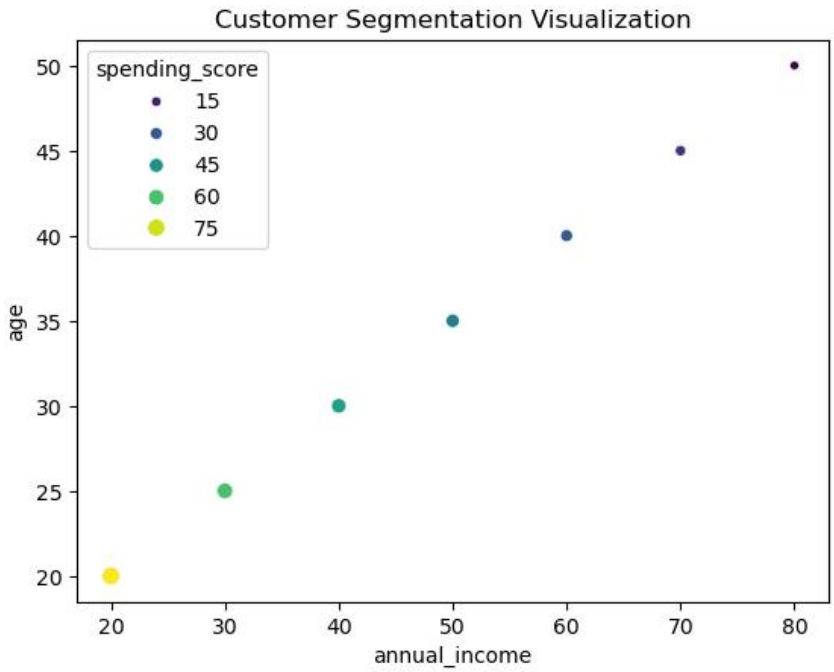
Write a program to:

- Visualize relationships using Seaborn scatter plots
- Color-code points based on spending score
- Identify potential customer segments visually

Code:

```
customers = pd.DataFrame({
    'age':[25,35,45,20,30,40,50],
    'annual_income':[30,50,70,20,40,60,80],
    'spending_score':[60,40,20,80,50,30,10]
})

sns.scatterplot(x='annual_income', y='age', hue='spending_score', size='spending_score', data=customers, palette='viridis')
plt.title("Customer Segmentation Visualization")
plt.show()
```

Output:



Q5. Model Result Visualization

You trained a classifier and obtained y_true and y_pred.

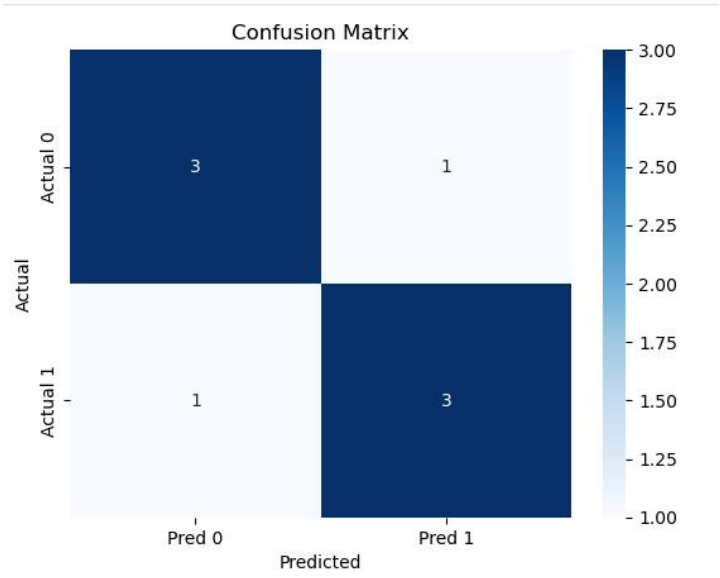
Write code using Seaborn to:

- Plot a confusion matrix heatmap
- Add annotations and color bar
- Clearly label predicted vs actual classes

Code:

```
from sklearn.metrics import confusion_matrix
import numpy as np

y_true = [0,1,0,1,0,1,1,0]
y_pred = [0,1,0,0,0,1,1,1]

cm = confusion_matrix(y_true, y_pred)

sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=['Pred 0','Pred 1'], yticklabels=['Actual 0','Actual 1'])
plt.title("Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```

Output:



Machine learning section

Q1. Telecom Customer Churn Prediction

Given customer data with features such as tenure, monthly_charges, and contract_type, and target churn.

Write a Scikit-learn program to:

- Encode categorical variables
- Train a Logistic Regression model
- Evaluate using accuracy and confusion matrix

Code:

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix
from sklearn.preprocessing import LabelEncoder

df = pd.DataFrame({
    'tenure':[1,5,10,2,8],
    'monthly_charges':[30,70,100,40,90],
    'contract_type':['Month-to-month','One year','Two year','Month-to-month','One year'],
    'churn':[1,0,0,1,0]
})

# Encode categorical
le = LabelEncoder()
df['contract_type'] = le.fit_transform(df['contract_type'])

X = df[['tenure','monthly_charges','contract_type']]
y = df['churn']

X_train,X_test,y_train,y_test = train_test_split(X,y,test_size=0.3,random_state=42)

model = LogisticRegression()
model.fit(X_train,y_train)
y_pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test,y_pred))
print("Confusion Matrix:\n", confusion_matrix(y_test,y_pred))
```

Output:

```
Accuracy: 0.5
Confusion Matrix:
 [[1 1]
 [0 0]]
```

Q2. House Price Prediction

A dataset contains area, bedrooms, location_score, and price.
Implement a regression model to:

- Split data into train/test sets
- Train Linear Regression
- Predict prices and calculate Mean Squared Error

Code:

```
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error

houses = pd.DataFrame({
    'area':[1000,1500,2000,1200,1800],
    'bedrooms':[2,3,4,2,3],
    'location_score':[7,8,9,6,8],
    'price':[200000,300000,400000,220000,350000]
})

X = houses[['area','bedrooms','location_score']]
y = houses['price']

X_train,X_test,y_train,y_test = train_test_split(X,y,test_size=0.3,random_state=42)

lr = LinearRegression()
lr.fit(X_train,y_train)
y_pred = lr.predict(X_test)

print("MSE:", mean_squared_error(y_test,y_pred))
```

Output:

```
MSE: 17817.665942533677
```

Q3. Disease Classification System

Patient data includes glucose, BMI, age, and outcome.

Write code to:

- Normalize the data
- Train a Decision Tree classifier
- Evaluate precision, recall, and F1-score

Code:

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import classification_report
from sklearn.preprocessing import StandardScaler

patients = pd.DataFrame({
    'glucose':[120,150,180,90,200],
    'BMI':[25,30,35,20,40],
    'age':[30,40,50,25,60],
    'outcome':[0,1,1,0,1]
})

X = patients[['glucose','BMI','age']]
y = patients['outcome']

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

X_train,X_test,y_train,y_test = train_test_split(X_scaled,y,test_size=0.3,random_state=42)

dt = DecisionTreeClassifier()
dt.fit(X_train,y_train)
y_pred = dt.predict(X_test)

print(classification_report(y_test,y_pred))
```

Output:

	precision	recall	f1-score	support
0	0.00	0.00	0.00	0
1	1.00	0.50	0.67	2
accuracy			0.50	2
macro avg	0.50	0.25	0.33	2
weighted avg	1.00	0.50	0.67	2

Q4. Email Spam Detection

You are given email text and spam labels.
Using Scikit-learn:

- Convert text to numerical features (TF-IDF)
- Train a Naive Bayes classifier
- Test the model on new email text

Code:

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB

emails = pd.DataFrame({
    'text':["Win money now","Meeting tomorrow","Cheap loans available","Project deadline"],
    'spam':[1,0,1,0]
})

vectorizer = TfidfVectorizer()
X = vectorizer.fit_transform(emails['text'])
y = emails['spam']

X_train,X_test,y_train,y_test = train_test_split(X,y,test_size=0.3,random_state=42)

nb = MultinomialNB()
nb.fit(X_train,y_train)
y_pred = nb.predict(X_test)

print("Accuracy:", accuracy_score(y_test,y_pred))

# Test new email
new_email = vectorizer.transform(["Exclusive offer just for you"])
print("Prediction (1=Spam,0=Not Spam):", nb.predict(new_email)[0])
```

Output:

```
Accuracy: 0.0
Prediction (1=Spam,0=Not Spam): 1
```

Q5. Student Risk Prediction

Student records include attendance, mid_marks, and final_marks.
Write a program to:

- Build a Random Forest classifier
- Predict whether a student is “At Risk”
- Display feature importance

Code:

```python
from sklearn.ensemble import RandomForestClassifier

students = pd.DataFrame({
    'attendance':[80,60,90,50,70],
    'mid_marks':[60,40,70,30,50],
    'final_marks':[70,50,80,40,60],
    'risk':['No','Yes','No','Yes','No']
})

X = students[['attendance','mid_marks','final_marks']]
y = students['risk']

X_train,X_test,y_train,y_test = train_test_split(X,y,test_size=0.3,random_state=42)

rf = RandomForestClassifier()
rf.fit(X_train,y_train)
y_pred = rf.predict(X_test)

print("Accuracy:", accuracy_score(y_test,y_pred))
print("Feature Importance:", rf.feature_importances_)
```

Output:

```
Accuracy: 1.0
Feature Importance: [0.35211268 0.30985915 0.33802817]
```

Lab No: 05

(TENSORFLOW IN PYTHON)

Where is TensorFlow used?
TensorFlow is used in:

- Image recognition
- Face detection
- Voice assistants
- Text classification
- Chatbots
- Medical analysis
- Recommendation systems (like Netflix)
- Self-driving cars

Code:

```
import tensorflow as tf
from tensorflow import keras

# Simple model with one hidden layer
model = keras.Sequential([
    keras.layers.Dense(10, activation='relu'),
    keras.layers.Dense(1)
])

# Compile the model
model.compile(optimizer='adam', loss='mse')

# Dummy data
x = [1, 2, 3, 4]
y = [2, 4, 6, 8]

# Train the model
model.fit(x, y, epochs=50)

# Predict
print(model.predict([5]))
```

2. How TensorFlow Works

TensorFlow works with **tensors**, which are just **multi-dimensional arrays**. It builds a **computation graph**, where each node represents a mathematical operation, and edges represent data (tensors) flowing between operations.

Key Components:

1. **Tensors:** Basic data structure (like arrays or matrices).
   Example: Scalars (0D), Vectors (1D), Matrices (2D), etc.
2. **Operations (Ops):** Mathematical operations applied to tensors (like addition, multiplication).
3. **Computational Graph:** A graph representing the flow of data through operations.
4. **Sessions (TF1.x) / Eager Execution (TF2.x):**
   - TensorFlow 1.x used sessions to run graphs.
   - TensorFlow 2.x executes operations immediately (like regular Python), making it easier to debug

Implementation:

```python
# Import necessary libraries
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt

# Step 1: Prepare Data
# Features (X) and labels (Y)
X = np.array([1, 2, 3, 4, 5], dtype=float)
Y = np.array([2, 4, 6, 8, 10], dtype=float)  # Linear relationship: y = 2*x

# Step 2: Build the Model
model = tf.keras.Sequential([
    tf.keras.layers.Dense(units=1, input_shape=[1])  # 1 input, 1 output
])

# Step 3: Compile the Model
model.compile(
    optimizer='sgd',               # Stochastic Gradient Descent optimizer
    loss='mean_squared_error'      # Loss function
)

# Step 4: Train the Model
history = model.fit(X, Y, epochs=1000, verbose=0)  # Train silently

# Step 5: Make Predictions
new_X = np.array([6, 7, 8], dtype=float)
predictions = model.predict(new_X)
print("Predictions for [6, 7, 8]:", predictions.flatten())

# Step 6: Visualize the Results
plt.scatter(X, Y, color='blue', label='Original Data')  # Original points
plt.plot(X, model.predict(X), color='red', label='Fitted Line')  # Regression line
plt.scatter(new_X, predictions, color='green', label='Predictions')  # Predictions
plt.xlabel("X")
plt.ylabel("Y")
plt.title("Linear Regression using TensorFlow")
plt.legend()
plt.show()
```

Output:

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:93: UserWarning: Do not pass an `input_shape`/`input_dim` argument
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 56ms/step
Predictions for [6, 7, 8]: [11.986539  13.9809065 15.975274 ]
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 53ms/step
```